🦾 Robotic Arm Forward and Inverse Kinematics

1. Course Overview

This course introduces the forward kinematics (FK) and inverse kinematics (IK) computation principles for the DOFBOT Pro 6-DOF robotic arm, C++ KDL solver implementation, and ROS 2 service invocation methods:

- **Forward Kinematics (FK)**: Input servo joint angles to calculate the pose of the end-effector gripper reference frame relative to the robotic arm base frame in 3D space
- **Inverse Kinematics (IK)**: Input the target position and orientation of the end-effector, optionally with current joint angles, to solve for the target angles of servos 1-5

---

2. Preparation and Running

This case suggests opening 3 terminals for step-by-step startup and debugging:

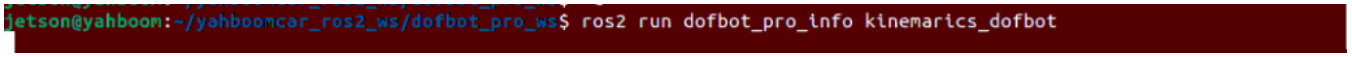
- **Terminal 1**: Start the kinematics solver server node (`kinemarics_arm`)
- **Terminal 2**: Start the robot URDF model, RViz visualization, and driver
- **Terminal 3**: Initiate ROS 2 service requests, publish control commands, and compare TF coordinate transforms

2.1 Start the Kinematics Solver Service

Run in **Terminal 1**:

```
ros2 run dofbot_pro_info kinemarics_dofbot
```

This node listens for `/get_kinemarics` service requests in the background after startup, with the terminal remaining on standby.



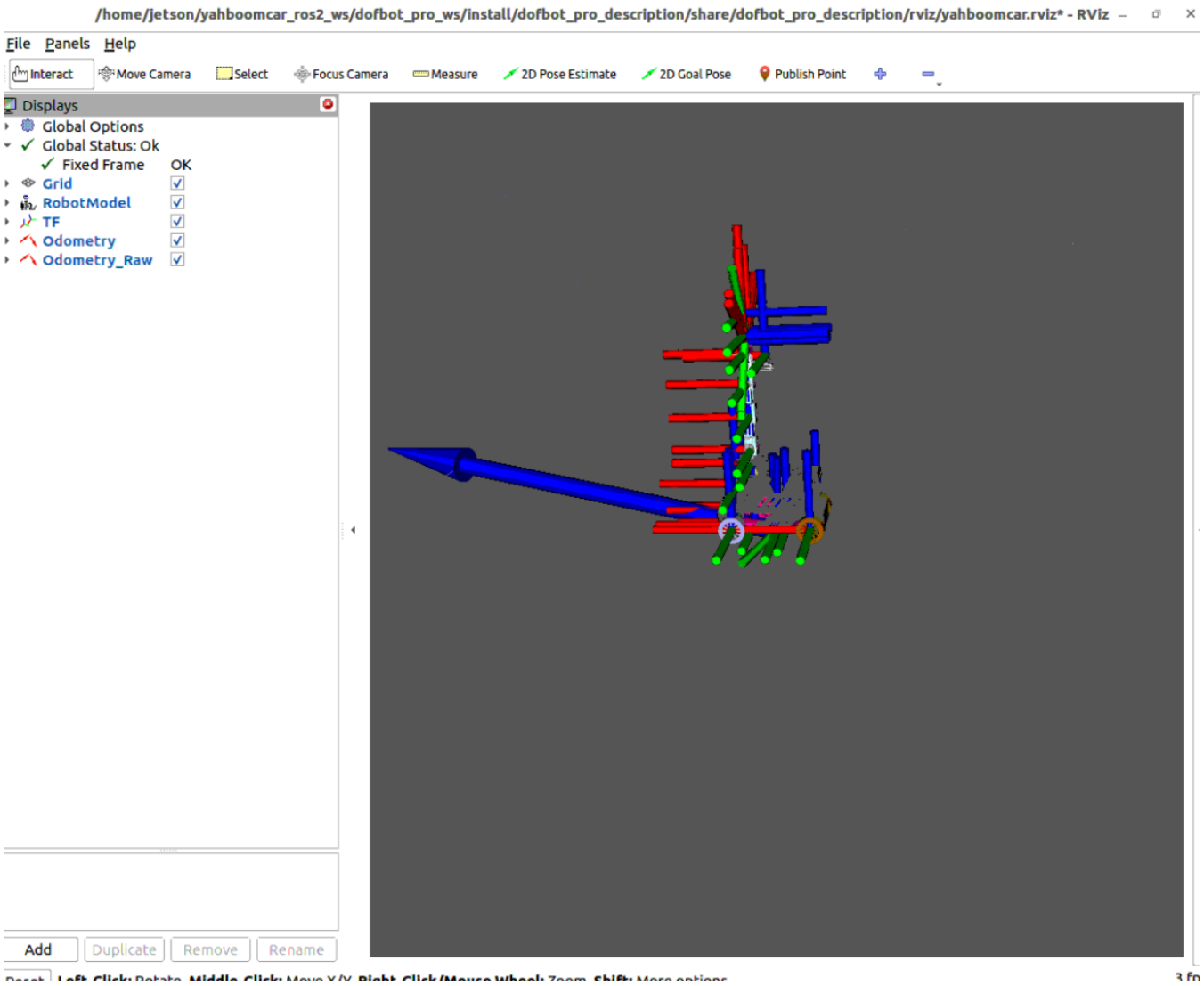
2.2 Start the Theoretical Model and RViz Visualization Environment

Run in **Terminal 2** (enable GUI joint slider mode):

```
ros2 launch dofbot_pro_description display_A1pro.launch.py gui:=true
```

In `gui:=true` mode, `/joint_states` is generated by the graphical sliders, and RViz renders the manually set theoretical model pose, facilitating comparison between KDL algorithm computed values and model TF.

```
jetson@yahboom:~$ ros2 launch dofbot_pro_description display_A1pro.launch.py \
  gui:=true fixed_frame:=arm_base_Link
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-09-29-28-799679-yahboom-59529
[INFO] [launch]: Default logging verbosity is set to INFO
--------------------robot_type = A1--------------------
[INFO] [joint_state_publisher_gui-1]: process started with pid [59585]
[INFO] [robot_state_publisher-2]: process started with pid [59587]
[INFO] [rviz2-3]: process started with pid [59589]
[INFO] [Ackman_driver_A1P-4]: process started with pid [59591]
[INFO] [base_node_A1-5]: process started with pid [59593]
[INFO] [imu_attitude_converter-6]: process started with pid [59595]
[base_node_A1-5] [INFO] [1788312569.050527675] [base_node]: A1 odometry: linear_scale_x=1.000000 angular_scale_z=1.000000 wheelbase_
compat=0.250 max_dt=0.500
[robot_state_publisher-2] [INFO] [1788312569.096555949] [robot_state_publisher]: got segment Gripping
[robot_state_publisher-2] [INFO] [1788312569.096849347] [robot_state_publisher]: got segment arm1
[robot_state_publisher-2] [INFO] [1788312569.096882117] [robot_state_publisher]: got segment arm2
[robot_state_publisher-2] [INFO] [1788312569.096895494] [robot_state_publisher]: got segment arm3
[robot_state_publisher-2] [INFO] [1788312569.096905831] [robot_state_publisher]: got segment arm4
[robot_state_publisher-2] [INFO] [1788312569.096915848] [robot_state_publisher]: got segment arm5
[robot_state_publisher-2] [INFO] [1788312569.096925257] [robot_state_publisher]: got segment arm_base_Link
[robot_state_publisher-2] [INFO] [1788312569.096934889] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1788312569.096944394] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1788312569.096954091] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1788312569.096963499] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-2] [INFO] [1788312569.096972844] [robot_state_publisher]: got segment laser_Link
[robot_state_publisher-2] [INFO] [1788312569.096982285] [robot_state_publisher]: got segment lbw_Link
[robot_state_publisher-2] [INFO] [1788312569.096991213] [robot_state_publisher]: got segment lfr_Link
[robot_state_publisher-2] [INFO] [1788312569.097000398] [robot_state_publisher]: got segment lfw_Link
[robot_state_publisher-2] [INFO] [1788312569.097009039] [robot_state_publisher]: got segment llink1
[robot_state_publisher-2] [INFO] [1788312569.097018287] [robot_state_publisher]: got segment llink2
[robot_state_publisher-2] [INFO] [1788312569.097027472] [robot_state_publisher]: got segment llink3
[robot_state_publisher-2] [INFO] [1788312569.097036529] [robot_state_publisher]: got segment rbw_Link
[robot_state_publisher-2] [INFO] [1788312569.097045842] [robot_state_publisher]: got segment rfr_Link
[robot_state_publisher-2] [INFO] [1788312569.097054674] [robot_state_publisher]: got segment rfw_Link
[robot_state_publisher-2] [INFO] [1788312569.097063507] [robot_state_publisher]: got segment rlink1
[robot_state_publisher-2] [INFO] [1788312569.097072115] [robot_state_publisher]: got segment rlink2
[robot_state_publisher-2] [INFO] [1788312569.097080884] [robot_state_publisher]: got segment rlink3
[joint_state_publisher_gui-1] [INFO] [1788312570.125347120] [joint_state_publisher]: Waiting for robot_description to be published o
n the robot_description topic...
[joint_state_publisher_gui-1] [INFO] [1788312570.142206001] [joint_state_publisher]: Centering
[imu_attitude_converter-6] [INFO] [1788312570.145792635] [imu_display_tf]: IMU attitude converter: /imu/data_raw -> /imu/data, publi
sh_tf=True
[joint_state_publisher_gui-1] [INFO] [1788312570.299860851] [joint_state_publisher]: Centering
[rviz2-3] [INFO] [1788312570.565697349] [rviz2]: Stereo is NOT SUPPORTED
[rviz2-3] [INFO] [1788312570.566228716] [rviz2]: OpenGl version: 4.5 (GLSL 4.5)
[Ackman_driver_A1P-4] [INFO] [1788312570.849984526] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[rviz2-3] [INFO] [1788312571.442078128] [rviz2]: Stereo is NOT SUPPORTED
```



2.3 Check Service and Node Status

Run in **Terminal 3**:

```
ros2 service list | grep get_kinemarics
ros2 node info /kinemarics_arm
```

```
jetson@yahboom:~$ ros2 service list | grep get_kinemarics
/get_kinemarics
jetson@yahboom:~$ ros2 node info /kinemarics_arm
/kinemarics_arm
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /get_kinemarics: arm_interface/srv/ArmKinemarics
    /kinemarics_arm/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /kinemarics_arm/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /kinemarics_arm/get_parameters: rcl_interfaces/srv/GetParameters
    /kinemarics_arm/list_parameters: rcl_interfaces/srv/ListParameters
    /kinemarics_arm/set_parameters: rcl_interfaces/srv/SetParameters
    /kinemarics_arm/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:

jetson@yahboom:~$
```

3. Service Interface and Data Structure

The kinematics node provides the `/get_kinemarics` service with type `arm_interface/srv/ArmKinemarics`:

```
ros2 interface show arm_interface/srv/ArmKinemarics
```

```
jetson@yahboom:~$ ros2 interface show arm_interface/srv/ArmKinemarics
float64 tar_x
float64 tar_y
float64 tar_z
float64 roll
float64 pitch
float64 yaw
float64 cur_joint1
float64 cur_joint2
float64 cur_joint3
float64 cur_joint4
float64 cur_joint5
float64 cur_joint6
string  kin_name
---
float64 joint1
float64 joint2
float64 joint3
float64 joint4
float64 joint5
float64 joint6
float64 x
float64 y
float64 z
float64 roll
float64 pitch
float64 yaw
jetson@yahboom:~$
```

3.1 Field Definitions and Mode Description

```
# ====== Request ======
float64 tar_x       # Target X coordinate (meters, used in IK mode)
float64 tar_y       # Target Y coordinate (meters, used in IK mode)
float64 tar_z       # Target Z coordinate (meters, used in IK mode)
float64 roll        # Target Roll angle (radians, used in IK mode)
float64 pitch       # Target Pitch angle (radians, used in IK mode)
float64 yaw         # Target Yaw angle (radians, used in IK mode)
float64 cur_joint1  # Current servo 1 angle (degrees, used in FK; used as IK seed value)
float64 cur_joint2  # Current servo 2 angle (degrees, used in FK; used as IK seed value)
float64 cur_joint3  # Current servo 3 angle (degrees, used in FK; used as IK seed value)
float64 cur_joint4  # Current servo 4 angle (degrees, used in FK; used as IK seed value)
float64 cur_joint5  # Current servo 5 angle (degrees, used in FK; used as IK seed value)
float64 cur_joint6  # Current gripper servo angle (degrees, not used in computation)
string  kin_name    # "fk" for forward mode, "ik" for inverse mode
---
# ====== Response ======
float64 joint1      # Servo 1 target angle computed by IK (degrees)
float64 joint2      # Servo 2 target angle computed by IK (degrees)
float64 joint3      # Servo 3 target angle computed by IK (degrees)
float64 joint4      # Servo 4 target angle computed by IK (degrees)
float64 joint5      # Servo 5 target angle computed by IK (degrees)
float64 joint6      # Gripper servo reserved (default 0)
float64 x           # End-effector X coordinate computed by FK (meters)
```

```
float64 y           # End-effector Y coordinate computed by FK (meters)
float64 z           # End-effector Z coordinate computed by FK (meters)
float64 roll        # End-effector Roll angle computed by FK (radians)
float64 pitch       # End-effector Pitch angle computed by FK (radians)
float64 yaw         # End-effector Yaw angle computed by FK (radians)
```

- `kin_name: 'fk'`: Reads `cur_joint1 ~ cur_joint5`, computes and returns `x, y, z, roll, pitch, yaw`;
- `kin_name: 'ik'`: Reads target pose `(tar_x, tar_y, tar_z, roll, pitch, yaw)` and current joint angles as seed values, computes and returns `joint1 ~ joint5`.

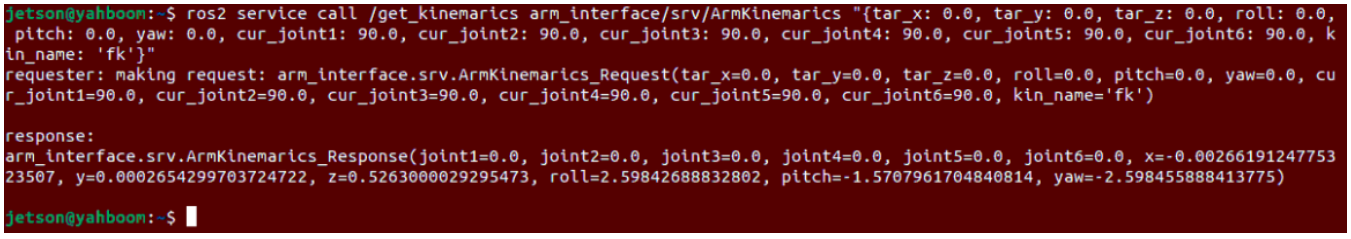
---

4. Theoretical Model Verification and TF Comparison

4.1 Forward Kinematics (FK) Call Verification

Keep all joints centered in the GUI (servo angles corresponding to 90 degrees for each joint), and initiate the forward kinematics service request in **Terminal 3**:

```
ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: 0.0, tar_y: 0.0, tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0, cur_joint2: 90.0, cur_joint3: 90.0, cur_joint4: 90.0, cur_joint5: 90.0, cur_joint6: 90.0, kin_name: 'fk'}"
```



The service returns the current end-effector pose approximately as:

```
x     = -0.0026619 m
y     =  0.0002654 m
z     =  0.5263000 m
roll  =  2.5984269 rad
pitch = -1.5707962 rad
yaw   = -2.5984559 rad
```

4.2 Inverse Kinematics (IK) Closed-Loop Verification

Fill the spatial pose obtained from forward kinematics into the inverse kinematics request to verify if it accurately back-calculates the initial servo angles [90, 90, 90, 90, 90]:

```
ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: -0.00266, tar_y: 0.000265, tar_z: 0.5263, roll: 2.598, pitch: -1.5707, yaw: -2.598, cur_joint1: 0.0, cur_joint2: 0.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 0.0, cur_joint6: 0.0, kin_name: 'ik'}"
```

```
jetson@yahboom:~$ ros2 service call /get_kinematics arm_interface/srv/ArmKinematics "{tar_x: -0.00266, tar_y: 0.000265, tar_z: 0.526
3, roll: 2.598, pitch: -1.5707, yaw: -2.598, cur_joint1: 0.0, cur_joint2: 0.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 0.0, cu
r_joint6: 0.0, kin_name: 'ik'}"
requester: making request: arm_interface.srv.ArmKinematics_Request(tar_x=-0.00266, tar_y=0.000265, tar_z=0.5263, roll=2.598, pitch=-
1.5707, yaw=-2.598, cur_joint1=0.0, cur_joint2=0.0, cur_joint3=0.0, cur_joint4=0.0, cur_joint5=0.0, cur_joint6=0.0, kin_name='ik')

response:
arm_interface.srv.ArmKinematics_Response(joint1=90.0, joint2=90.0, joint3=90.0, joint4=90.0, joint5=90.0, joint6=0.0, x=0.0, y=0.0,
z=0.0, roll=0.0, pitch=0.0, yaw=0.0)

jetson@yahboom:~$
```

4.3 TF Transform Ground Truth Comparison

FK computes the relative transform between `arm_base_Link -> Gripping` coordinate frames. Use the TF tool to print and compare values:

```
ros2 run tf2_ros tf2_echo arm_base_Link Gripping
```

```
jetson@yahboom:~$ ros2 run tf2_ros tf2_echo arm_base_Link Gripping
[INFO] [1788271402.933322631] [tf2_echo]: Waiting for transform arm_base_Link ->  Gripping
: Invalid frame ID "arm_base_Link" passed to canTransform argument target_frame - frame do
es not exist
At time 1788271403.857279895
- Translation: [-0.003, 0.000, 0.526]
- Rotation: in Quaternion (xyzw) [-0.000, -0.707, -0.000, 0.707]
- Rotation: in RPY (radian) [2.598, -1.571, -2.598]
- Rotation: in RPY (degree) [148.879, -90.000, -148.881]
- Matrix:
 -0.000  0.000 -1.000 -0.003
 -0.000  1.000  0.000  0.000
  1.000  0.000 -0.000  0.526
  0.000  0.000  0.000  1.000
At time 1788271404.857405757
- Translation: [-0.003, 0.000, 0.526]
- Rotation: in Quaternion (xyzw) [-0.000, -0.707, -0.000, 0.707]
- Rotation: in RPY (radian) [2.598, -1.571, -2.598]
- Rotation: in RPY (degree) [148.879, -90.000, -148.881]
- Matrix:
 -0.000  0.000 -1.000 -0.003
 -0.000  1.000  0.000  0.000
  1.000  0.000 -0.000  0.526
  0.000  0.000  0.000  1.000
^C[INFO] [1788271405.611558577] [rclcpp]: signal_handler(signum=2)
jetson@yahboom:~$
```

The Translation values in TF are completely consistent with FK's `(x, y, z)`.

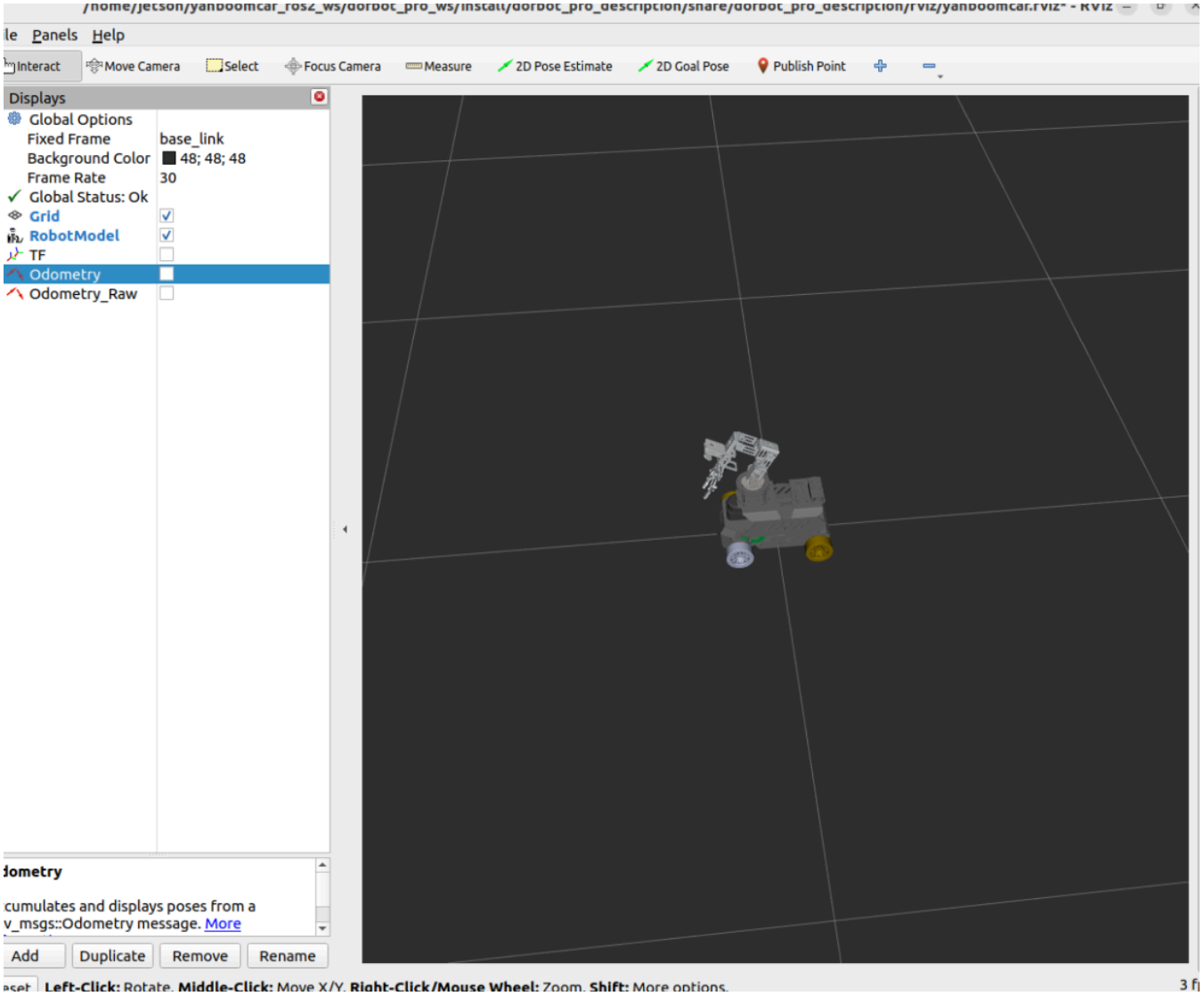
---

5. Physical Robotic Arm Motion and Physical Closed-Loop Verification

After completing theoretical verification, stop the GUI simulation in **Terminal 2** with `Ctrl+C` and switch to physical robot real feedback mode:

```
ros2 launch dofbot_pro_description display_A1pro.launch.py gui:=false
```

At this point, the driver node reads physical servo angles in real-time and publishes `/arm_status/arm_joint`, and RViz and the TF tree faithfully reflect the real robotic arm pose.



5.1 Drive the Robotic Arm to a Specified Working Pose

Publish multi-joint control commands in **Terminal 3**:

```
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 120,
joint3: 0, joint4: 0, joint5: 90, joint6: 90, time: 1500}" --once
```

```
jetson@yahboom:~$ ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, j
oint2: 120, joint3: 0, joint4: 0, joint5: 90, joint6: 90, time: 1500}" --once
publisher: beginning loop
publishing #1: dofbot_pro_interface.msg.ArmJoints(joint1=90, joint2=120, joint3=0, joint4=0, joi
nt5=90, joint6=90, time=1500)

jetson@yahboom:~$
```

5.2 Get End-Effector Pose Based on Current Actual Joint Angles (FK)

```
ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: 0.0, tar_y: 0.0,
tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0, cur_joint2: 120.0,
cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 90.0, cur_joint6: 90.0, kin_name: 'fk'}"
```

Obtain the end-effector pose:

```
jetson@yahboom:~$ ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: 0.0, tar_y: 0.0, tar_z: 0
.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0, cur_joint2: 120.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5
: 90.0, cur_joint6: 90.0, kin_name: 'fk'}"
requester: making request: arm_interface.srv.ArmKinemarics_Request(tar_x=0.0, tar_y=0.0, tar_z=0.0, roll=0.0, pitch=0.
0, yaw=0.0, cur_joint1=90.0, cur_joint2=120.0, cur_joint3=0.0, cur_joint4=0.0, cur_joint5=90.0, cur_joint6=90.0, kin_n
ame='fk')

response:
arm_interface.srv.ArmKinemarics_Response(joint1=0.0, joint2=0.0, joint3=0.0, joint4=0.0, joint5=0.0, joint6=0.0, x=0.1
1178395298285 34, y=0.000262361564850923, z=0.1602151378828678, roll=-3.610409982618074e-06, pitch=1.04719739710221, ya
w=-2.744163662914015e-05)

jetson@yahboom:~$
```

5.3 Back-Calculate Servo Angles Based on Obtained Pose (IK)

```
ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: 0.11178, tar_y:
0.000262, tar_z: 0.16021, roll: -3.61e-06, pitch: 1.04719, yaw: -2.74e-05, cur_joint1: 90.0,
cur_joint2: 120.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 90.0, cur_joint6: 90.0,
kin_name: 'ik'}"
```

Inverse solved joint angles:

```
jetson@yahboom:~$ ros2 service call /get_kinemarics arm_interface/srv/ArmKinemarics "{tar_x: 0.1117839529828534, tar_y
: 0.000262361564850923, tar_z: 0.1602151378828678, roll: -3.610409982618074e-06, pitch: 1.04719739710221, yaw: -2.7441
63662914015e-05, cur_joint1: 0.0, cur_joint2: 0.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 0.0, cur_joint6: 0.0,
 kin_name: 'ik'}"
requester: making request: arm_interface.srv.ArmKinemarics_Request(tar_x=0.1117839529828534, tar_y=0.00026236156485092
3, tar_z=0.1602151378828678, roll=-3.610409982618074e-06, pitch=1.04719739710221, yaw=-2.744163662914015e-05, cur_join
t1=0.0, cur_joint2=0.0, cur_joint3=0.0, cur_joint4=0.0, cur_joint5=0.0, cur_joint6=0.0, kin_name='ik')

response:
arm_interface.srv.ArmKinemarics_Response(joint1=89.999999817571, joint2=119.97286182219285, joint3=0.026377079175347262
, joint4=-0.012581786699854547, joint5=89.99998049516324, joint6=0.0, x=0.0, y=0.0, z=0.0, roll=0.0, pitch=0.0, yaw=0
.0)

jetson@yahboom:~$
```

5.4 View Physical Robot TF Transform

```
ros2 run tf2_ros tf2_echo arm_base_Link Gripping
```

Verify the real transform correspondence:

```
jetson@yahboom:~$ ros2 run tf2_ros tf2_echo arm_base_Link Gripping
[INFO] [1788313680.464612764] [tf2_echo]: Waiting for transform arm_base_Link ->  Gripping: Invalid frame ID "arm_base
_Link" passed to canTransform argument target_frame - frame does not exist
At time 1788313681.437586597
- Translation: [0.111, 0.000, 0.158]
- Rotation: in Quaternion (xyzw) [0.000, 0.508, -0.000, 0.862]
- Rotation: in RPY (radian) [-0.000, 1.065, -0.000]
- Rotation: in RPY (degree) [-0.000, 61.000, -0.002]
- Matrix:
  0.485  0.000  0.875  0.111
 -0.000  1.000 -0.000  0.000
 -0.875 -0.000  0.485  0.158
  0.000  0.000  0.000  1.000
At time 1788313682.437419586
- Translation: [0.111, 0.000, 0.158]
- Rotation: in Quaternion (xyzw) [0.000, 0.508, -0.000, 0.862]
- Rotation: in RPY (radian) [-0.000, 1.065, -0.000]
- Rotation: in RPY (degree) [-0.000, 61.000, -0.002]
- Matrix:
  0.485  0.000  0.875  0.111
 -0.000  1.000 -0.000  0.000
 -0.875 -0.000  0.485  0.158
  0.000  0.000  0.000  1.000
At time 1788313683.437613210
- Translation: [0.111, 0.000, 0.158]
- Rotation: in Quaternion (xyzw) [0.000, 0.508, -0.000, 0.862]
- Rotation: in RPY (radian) [-0.000, 1.065, -0.000]
- Rotation: in RPY (degree) [-0.000, 61.000, -0.002]
- Matrix:
  0.485  0.000  0.875  0.111
 -0.000  1.000 -0.000  0.000
 -0.875 -0.000  0.485  0.158
  0.000  0.000  0.000  1.000
```